



Facultad
de Matemáticas
UMU



Memoria de Prácticas Externas

Procesamiento de Señal Radar

Autor: María Heras Rico

Tutor de prácticas: Francisco Salmerón Yuste

Empresa: A4Radar: Advanced Algorithms For Radar

Índice

1. A4Radar	2
2. Descripción del puesto	3
2.1. Descubrimiento empresa	3
2.2. Descripción del puesto	3
2.3. Presentación	3
2.4. Organización	3
3. Entorno Tecnológico y Herramientas	4
3.1. Lenguaje de Programación y Adaptación	4
3.2. Librerías de Procesamiento y Cálculo Matemático	4
3.3. Herramientas de Visualización	4
3.4. Formato de Datos y Hardware	4
4. Desarrollo	5
4.1. Primera semana: <i>Formación Inicial y Fundamentos</i>	5
4.2. Segunda semana: <i>Primera tarea</i>	5
4.3. Segunda tarea: Limpieza de señal	7
4.4. Tercera tarea: <i>Detección automática y precisión</i>	13
4.4.1. Algoritmo CFAR (<i>Constant False Alarm Rate</i>)	13
4.4.2. Precisión Sub-bin e Interpolación Gaussiana	15
4.5. Cuarta tarea: <i>Análisis de Micro-Doppler y Fases de Vibración</i>	16
4.5.1. La Fase de la Señal Compleja	16
4.5.2. Implementación: Desenrollado (Unwrap) y Estabilización	16
4.5.3. Última tarea: <i>Filtro de Kalman</i>	19
4.5.4. ¿Qué es el Filtro de Kalman y por qué es necesario?	19
5. Relación con el Grado	22
5.1. Álgebra Lineal y Geometría	22
5.2. Análisis Matemático y Variable Compleja	22
5.3. Estadística, Probabilidad y Optimización	22
5.4. Programación Estructurada y Algoritmia	23
6. Conclusiones	23

1. A4Radar

A4Radar es una empresa tecnológica española especializada en el desarrollo de soluciones innovadoras basadas en tecnología de radar de ondas milimétricas (mmWave) y banda ultraancha (UWB). Desde su creación, la compañía se ha posicionado como una empresa Deep Tech de referencia, actuando como fabricante de diseño original (ODM) para ofrecer sensores de alta precisión, bajo consumo y gran robustez. Su actividad se centra en democratizar el uso de sistemas radar en múltiples sectores estratégicos, tales como la Industria 4.0, las ciudades inteligentes y logística (Smart Cities), la gestión portuaria avanzada (Ports 4.0), el sector automovilístico y la monitorización biomédica (eHealth).

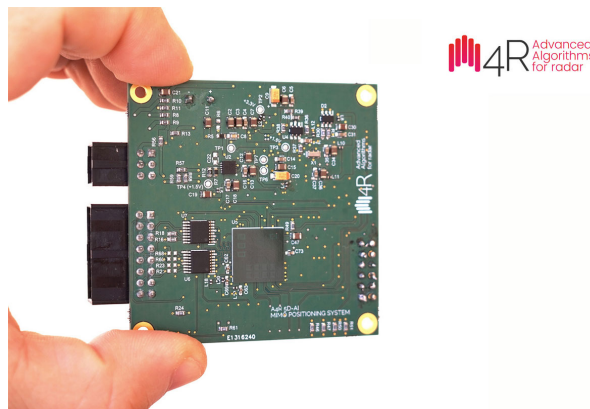


Figura 1: Logotipo representativo de la tecnología radar MIMO (Multiple-Input Multiple-Output).

A lo largo de su trayectoria, A4Radar ha desarrollado un modelo de negocio altamente especializado y versátil, caracterizado por el diseño y la fabricación europea de sus propios dispositivos electrónicos, así como del software embebido y los algoritmos de procesamiento de señal (incluyendo inteligencia artificial y deep learning). Esta integración total les permite ofrecer desde productos estandarizados hasta soluciones completamente personalizadas, adaptándose a los retos y requisitos más exigentes de cada cliente. Su capacidad para reducir la complejidad técnica y el tiempo de llegada al mercado (time-to-market) ha facilitado su participación en proyectos tecnológicos avanzados, estableciendo sólidas alianzas con entidades públicas, universidades y proveedores líderes de semiconductores (SoC).

Los servicios y productos de A4Radar se fundamentan en la excelencia técnica, la innovación continua y la aplicación de tecnología de vanguardia. Formada por un equipo multidisciplinar que aúna una profunda experiencia en I+D y gestión empresarial, la compañía apuesta por un enfoque centrado en el cliente. En un contexto global que demanda una rápida transformación digital y la automatización de procesos, A4Radar proporciona aplicaciones operativas en tiempo real que generan un gran valor añadido, permitiendo a empresas de todo tipo aprovechar las oportunidades de la tecnología radar sin importar su grado de conocimiento previo en hardware o procesamiento de señales.

2. Descripción del puesto

2.1. Descubrimiento empresa

Desde un principio tenía el claro objetivo de realizar mis prácticas curriculares en el sector privado, ya que es el entorno donde deseo desarrollarme profesionalmente en el futuro. Tras realizar una búsqueda entre las diferentes ofertas y los acuerdos de la universidad de años anteriores, descubrí A4Radar. Su perfil me llamó profundamente la atención por su capacidad para plantear soluciones innovadoras ante diversos retos tecnológicos. En lugar de restringirse a las aplicaciones clásicas del radar, como el control de velocidad en carreteras, exploran casos de uso de gran valor social, como biosensores remotos para el sector salud.

2.2. Descripción del puesto

Mis prácticas se han centrado desde el inicio en la inmersión total en la tecnología radar. El objetivo principal de mi puesto era recibir una formación integral que me dotara de todas las herramientas teóricas y prácticas necesarias para, en un futuro, poder integrarme profesionalmente en este sector tan especializado. Para ello, mis primeras tareas se centraron en aprender a extraer, leer e interpretar señales electromagnéticas en crudo (datos I/Q). Además, se me capacitó para aplicar el razonamiento matemático en la depuración del código, permitiéndome realizar todo tipo de ajustes algorítmicos orientados a limpiar el ruido, optimizar el rendimiento y mejorar significativamente la precisión en las tareas de detección y seguimiento.

2.3. Presentación

La primera toma de contacto con la empresa fue una entrevista en la que mi tutor, Francisco, me explicó a qué se dedica A4Radar y los diferentes proyectos que podíamos realizar. Pactamos empezar las prácticas en el segundo cuatrimestre y comenzar mi formación.

2.4. Organización

Las prácticas transcurrieron desde el 19 de enero hasta el 2 de marzo. Acudiría un par de veces por semana a la oficina y completaría las 120 horas con teletrabajo desde casa. Desde el primer momento Francisco se mostró totalmente flexible con mi horario. Y me dio total libertad para poder organizarme y compaginar los estudios de la manera que más me beneficiase.

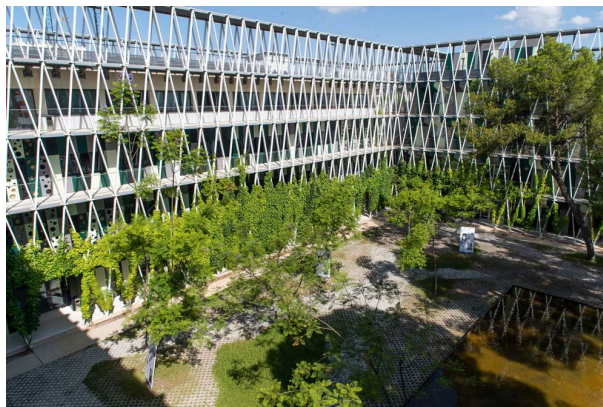


Figura 2: Parque Científico de Murcia. Localización oficina de A4Radar

3. Entorno Tecnológico y Herramientas

Para llevar a cabo el análisis, procesamiento y visualización de las señales de radar, se ha empleado un ecosistema de desarrollo basado principalmente en software libre y herramientas de análisis de datos de alto rendimiento. A continuación, detallaré las principales tecnologías utilizadas:

3.1. Lenguaje de Programación y Adaptación

El núcleo del proyecto se ha desarrollado íntegramente en **Python**. El dominio de un lenguaje de programación era un requisito técnico fundamental para poder afrontar este trabajo. Durante las reuniones iniciales, mi tutor se interesó por mi base en programación; los conocimientos de Python adquiridos durante las asignaturas del Grado, sumados a la base en C y C++ que obtuve durante mi estancia Erasmus, resultaron ser un factor clave para poder desempeñar la tarea.

Contar con esta sólida base algorítmica académica hizo que retomar la programación y adaptarme al entorno de trabajo fuera un proceso muy ágil y natural. Trabajar en Python ha sido una enorme ventaja operativa, debido a sus extensas librerías orientadas al cálculo científico, el procesamiento de señales y la ingeniería de datos.

3.2. Librerías de Procesamiento y Cálculo Matemático

Para implementar los algoritmos matemáticos, hice uso de las siguientes librerías:

- **NumPy**: Fundamental para el manejo eficiente de vectores y matrices multidimensionales. Se utilizó intensivamente para la manipulación de los datos I/Q extraídos del radar y para el cálculo de la Transformada Rápida de Fourier (FFT), operaciones de álgebra lineal y el desarrollo matricial del Filtro de Kalman.
- **SciPy**: Empleada para el procesamiento avanzado de señales. Funciones como *find_peaks* permitieron la detección de máximos en el algoritmo CFAR, mientras que utilidades como *detrend* y las ventanas de *Nuttall* fueron claves para aislar la vibración micrométrica (Micro-Doppler) de los vehículos y barcos.
- **Pandas**: Utilizada en las fases iniciales del flujo de trabajo para estructurar y filtrar los paquetes de datos binarios (formato PCAP/PCAPNG) provenientes de la red del radar en DataFrames manejables.

3.3. Herramientas de Visualización

Dado el volumen de información procesada, la representación gráfica fue esencial para validar los modelos matemáticos:

- **Plotly**: Se optó por esta librería gráfica interactiva para generar "dashboards" en formato HTML. Esto permitió renderizar simultáneamente gráficas de seguimiento (Tracking), mapas de calor bidimensionales (Range-Doppler 2D) y el espectro de fase desenrollada (*unwrapped phase*), facilitando el análisis visual de las trayectorias y vibraciones en tiempo real.

3.4. Formato de Datos y Hardware

Los datos crudos de entrada se proporcionaron en archivos de captura de red **PCAP / PCAPNG**. Estos archivos contenían la telemetría en bruto generada por los sensores FMCW (Frequency-Modulated Continuous-Wave) desarrollados por A4Radar, operando con diferentes resoluciones y configuraciones según el escenario (tráfico fluido o entorno portuario).

4. Desarrollo

4.1. Primera semana: *Formación Inicial y Fundamentos*

El periodo de prácticas comenzó con una fase de formación autodidacta intensiva, supervisada por mi tutor. Primero asentaba el material y después, me daba todas las explicaciones sobre el tema. Para adentrarme en el trabajo de la empresa y empezar a asimilar los conceptos necesarios para mis tareas, Francisco me mandó una serie de vídeos explicativos de nociones físicas básicas del radar, la tecnología **FMCW (Frequency-Modulated Continuous-Wave)**.

El proceso formativo se apoyó en materiales técnicos de referencia, destacando las sesiones formativas de *Texas Instruments* sobre sensores mmWave y los vídeos especializados de *Marshall Brun*, centrados en el procesamiento digital de señales aplicadas a radar.

Conseguí profundizar y entender los conceptos siguientes:

- La generación del chirp y el procesado de la señal de batido.
- La resolución en distancia y velocidad.
- La interpretación de datos complejos (I/Q).

4.2. Segunda semana: *Primera tarea*

Una vez asentados todos los conceptos básicos, me mandaron mi primera tarea. Me proporcionaron una carpeta, `calcula_fft`, que contenía una grabación simple de un radar frente a un único objetivo. El entorno constaba de un script base (`exporta_pcap`) para la lectura del formato de red, un visualizador tridimensional básico (`3d_visualizer`) y un código incompleto que debía modificar para procesar la grabación y extraer resultados medibles.

El objetivo principal era asimilar la esencia del radar **FMCW**: transformar la señal del dominio del tiempo al dominio del espacio. Para ello, tuve que familiarizarme con:

- **Señales I/Q (Fase y Cuadratura):** La comprensión de que el radar digitaliza las ondas en números complejos ($A + bj$), lo que permite conservar la información de la fase, vital para los cálculos de velocidad y vibración posteriores.

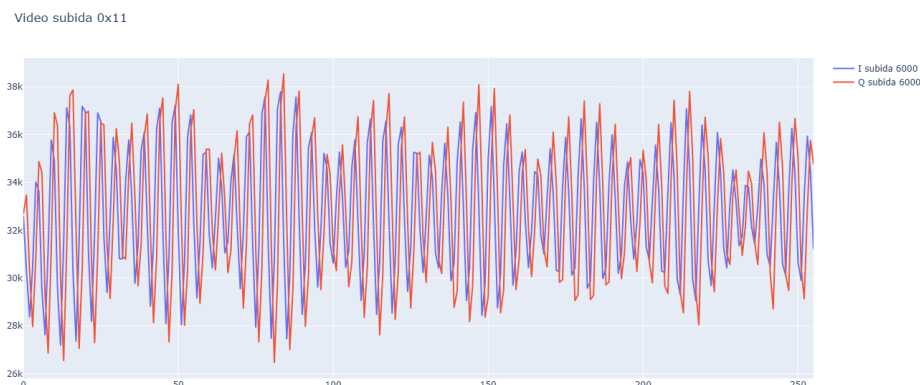


Figura 3: Subida I/Q.

- **La Transformada de Fourier (FFT):** El algoritmo matemático núcleo del proyecto, utilizado para descomponer la "frecuencia de batido" de la interferencia de la onda y convertirla en la distancia exacta al objetivo (medida en "baldosas." *bins*).

- **Decibelios (dB):** La necesidad de transformar la escala lineal de potencia a una escala logarítmica ($20 \log_{10}$) para compensar la drástica atenuación de la señal electromagnética con la distancia.

La prioridad fue lograr una ejecución funcional. El código desarrollado se limitó a realizar una transformación de Fourier directa y en bruto sobre los datos I/Q extraídos, sin aplicar aún ningún tipo de preprocesado, limpieza o relleno espectral.

```

1 for i in range(len(matriz_up_i)):
2     for j in range(len(matriz_up_i)):
3         signal_up = matriz_up_i[j] + 1j * matriz_up_q[j]
4         signal_down = matriz_down_i[j] + 1j * matriz_down_q[j]
5
6         # calculo fft
7         fft_result_up = np.fft.fft(signal_up, n=Nfft)
8         fft_result_down = np.fft.fft(signal_down, n=Nfft)
9
10        # pasamos a decibelios
11        # valor absoluto para ver la potencia
12        mag_up = 20 * np.log10(np.abs(fft_result_up) + 1e-9)
13        mag_down = 20 * np.log10(np.abs(fft_result_down) + 1e-9)
14
15        matriz_FFT.append(mag_up) # Guardamos la de subida como referencia
principal
16        matriz_FFT_up.append(mag_up)
17        matriz_FFT_down.append(mag_down)
18
19        # calculo fase para ver velocidad exacta
20        phase.append(np.angle(fft_result_up))

```

Listing 1: Cálculo de la FFT y obtención de potencia y fase.

Aunque este primer código logró ejecutarse y ubicar el objetivo, el análisis crítico de la gráfica resultante reveló tres graves deficiencias que evidenciaron la necesidad de procesamiento avanzado:

1. **El pico de la antena:** Se observó una saturación colosal en el metro cero que enmascaraba cualquier objeto cercano. Esto se debía al acoplamiento de corriente continua (*DC Offset*) en la electrónica de la placa.
2. **Falta de nitidez:** El pico del objeto no era una línea fina y exacta, sino una montaña ancha y borrosa cuya energía se desbordaba a los metros adyacentes, lo que imposibilitaba distinguir dos objetos juntos.
3. **Resolución pixelada:** Al no añadir puntos artificiales al cálculo de Fourier (*Zero-Padding*), la gráfica se veía fragmentada y angulosa.

También cabe destacar que al intentar generar y exportar las imágenes visuales de los resultados (como la FFT o el mapa tridimensional), surgió un bloqueo crítico en el entorno de desarrollo: el ordenador se colgaba de manera constante, paralizando la ejecución del programa.

Tras una depuración exhaustiva del código, el problema no residía en el algoritmo de cálculo en sí, sino en la gestión de la memoria RAM. Python y la librería NumPy operan por defecto con una precisión de punto flotante de 64 bits (float64). Al trabajar con grabaciones de radar de alta frecuencia, el volumen de datos de las matrices I/Q era tan masivo que, al intentar pasarlo a las librerías de renderizado gráfico, la memoria del sistema colapsaba.

Para resolverlo, optimicé la gestión de los tipos de datos en todo el código. Implementé un "downcasting" forzado durante la extracción de datos y el ventaneado, obligando al sistema a trabajar en 32 bits mediante el comando `.astype(np.float32)`.

Esta sencilla modificación matemática redujo el consumo de memoria RAM exactamente a la mitad. Al hacerlo, el renderizado de las imágenes y mapas de calor interactivos pasó a ejecutarse de forma fluida, sin que la pérdida de precisión decimal (de 64 a 32 bits) afectara en absoluto a la física o a los resultados finales de las distancias del radar.

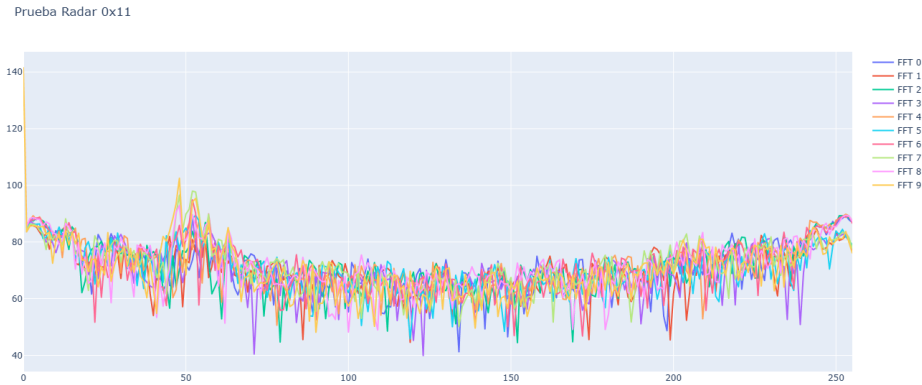


Figura 4: Resultado inicial de la FFT en bruto. Se observa la saturación masiva en el origen (metro 0) y el ensanchamiento del pico del objeto debido al *spectral leakage*.

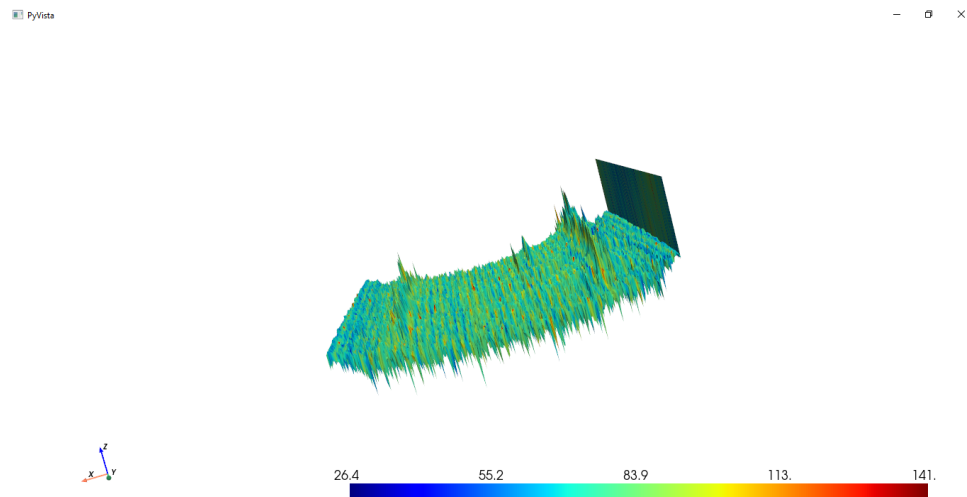


Figura 5: FFT en 3D.

Estos resultados fueron los desencadenantes de mi próxima tarea: la implementación de filtros de limpieza en el dominio del tiempo antes de ejecutar la FFT.

4.3. Segunda tarea: Limpieza de señal

La segunda fase del proyecto se centró en implementar técnicas de acondicionamiento de la señal en el dominio del tiempo, paso previo fundamental para asegurar la viabilidad de cualquier algoritmo de detección posterior.

Para resolver las deficiencias detectadas, se aplicaron tres conceptos clave del procesamiento digital de señales:

- **DC Removal** (Eliminación de Componente Continua): Las imperfecciones electrónicas del hardware del radar (acoplamiento entre la antena transmisora y receptora) generan un voltaje constante o componente de frecuencia cero”. Si no se elimina, la FFT lo interpreta

como un objeto gigantesco en el metro 0 (el "pico de la antena"). La solución matemática consiste en calcular el valor medio de la señal y restarlo a todos los puntos, centrando así la onda en el origen (cero voltios).

```

1      # 1. QUITAR LA MEDIA
2      #quitamos el pico que sale en dist 0, restamos la media, centramos
3      la se al en 0 y el pico desaparece
4      signal_up = signal_up - np.mean(signal_up)
5      signal_down = signal_down - np.mean(signal_down)
6
7      signal_up_g = signal_up - np.mean(signal_up) #para comparar con una
      sin media

```

Listing 2: Código DC Removal

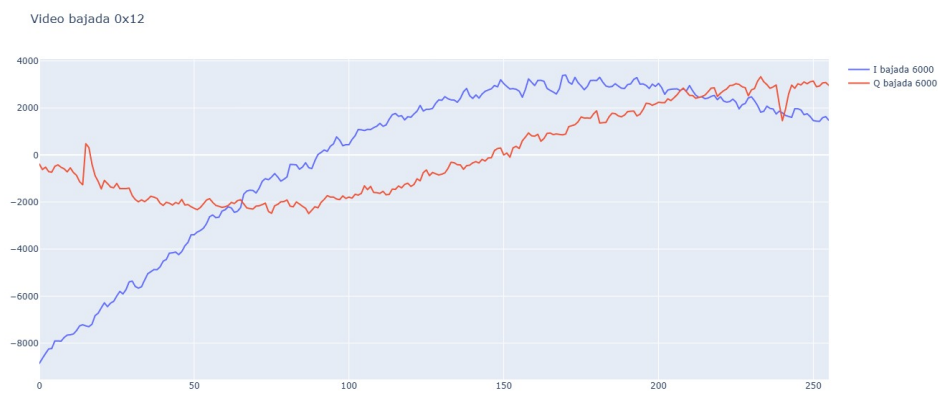


Figura 6: Bajada media.



Figura 7: FFT 2D.

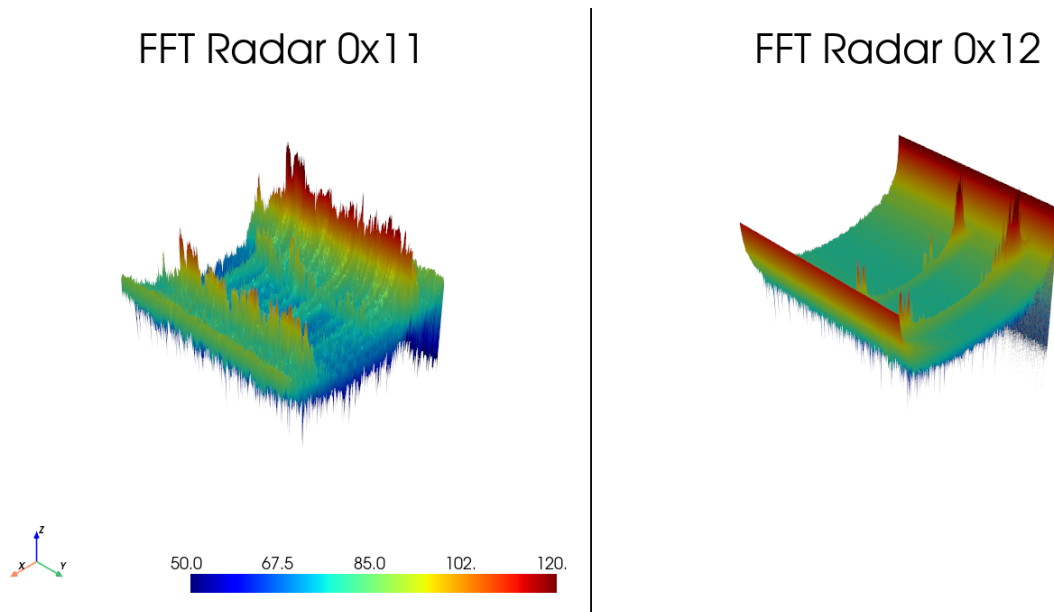


Figura 8: Comparación FFT 3D Sin Media Quitada/ Con Media Quitada.

- **Ventaneado:** La señal capturada por el radar no es infinita; está “cortada” bruscamente al inicio y al final del barrido (*chirp*). La FFT asume erróneamente que esta señal se repite, lo que genera frecuencias inexistentes que ensanchan los picos reales (*spectral leakage*). Al multiplicar la señal por una función de ventana (durante las prácticas probé numerosos tipos de ventanas para ver el efecto que hacían cada una de ellas y poder elegir la que mejor se adaptase al resultado buscado), atenúamos suavemente los extremos de la onda hacia cero. Esto estrecha significativamente los picos en la gráfica de distancias.

```

1 # Opciones ventanas
2 # 'rectangular'      -> Sin ventana (maxima resolucio, mucho ruido)
3 # 'triangular'      -> Simple, reduce algo de ruido.
4 # 'barlett'         -> Triangular estricta (cero en los bordes).
5 # 'hanning'         -> buen equilibrio.
6 # 'hamming'         -> Similar a Hanning pero no baja a cero en bordes.
7 # 'blackman'        -> Muy buena reduccion de ruido.
8 # 'blackman-harris' -> EXCELENTE para borrar ruido (sidelobes muy bajos).
9 # 'blackman-nuttall' -> Aun mas agresiva contra el ruido.
10 # 'flat-top'        -> Para medir la amplitud exacta (el pico es plano
    arriba).
11 # 'gauss'           -> Forma de campana natural.
12 # 'barlett-hann'    -> Hibrido entre Barlett y Hanning.
13
14 def generar_ventana(tipo, N):
15
16     n = np.arange(N)
17
18     if tipo == 'rectangular':
19         return np.ones(N)
20
21     elif tipo == 'triangular':
22         return 1 - np.abs((n - (N-1)/2) / (N/2))
23
24     elif tipo == 'barlett':
25         return np.bartlett(N)
26
27     elif tipo == 'hanning':
28         return np.hanning(N)

```

```

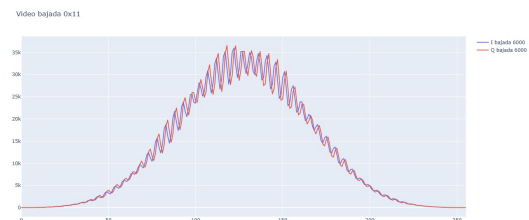
29
30 elif tipo == 'hamming':
31     return np.hamming(N)
32
33 elif tipo == 'blackman':
34     return np.blackman(N)
35
36 elif tipo == 'blackman-harris':
37     a0 = 0.35875; a1 = 0.48829; a2 = 0.14128; a3 = 0.01168
38     term1 = a1 * np.cos(2 * np.pi * n / (N - 1))
39     term2 = a2 * np.cos(4 * np.pi * n / (N - 1))
40     term3 = a3 * np.cos(6 * np.pi * n / (N - 1))
41     return a0 - term1 + term2 - term3
42
43 elif tipo == 'blackman-nuttall':
44     a0 = 0.3635819; a1 = 0.4891775; a2 = 0.1365995; a3 = 0.0106411
45     term1 = a1 * np.cos(2 * np.pi * n / (N - 1))
46     term2 = a2 * np.cos(4 * np.pi * n / (N - 1))
47     term3 = a3 * np.cos(6 * np.pi * n / (N - 1))
48     return a0 - term1 + term2 - term3
49
50 elif tipo == 'flat-top':
51     a0 = 1; a1 = 1.93; a2 = 1.29; a3 = 0.388; a4 = 0.032
52     term1 = a1 * np.cos(2 * np.pi * n / (N - 1))
53     term2 = a2 * np.cos(4 * np.pi * n / (N - 1))
54     term3 = a3 * np.cos(6 * np.pi * n / (N - 1))
55     term4 = a4 * np.cos(8 * np.pi * n / (N - 1))
56     return a0 - term1 + term2 - term3 + term4
57
58 elif tipo == 'gauss':
59     sigma = 0.4
60     return np.exp(-0.5 * ((n - (N - 1) / 2) / (sigma * (N - 1) / 2))
61 **2)
62
63 elif tipo == 'barlett-hann':
64     return 0.62 - 0.48 * np.abs(n / (N - 1) - 0.5) - 0.38 * np.cos(2 *
65 np.pi * (n / (N - 1)))
66
67 else:
68     print(f"AVISO: Ventana '{tipo}' no reconocida. Usando Rectangular.")
69     return np.ones(N)

```

Listing 3: Distintas ventanas posibles y sus propiedades



(a) Subida I/Q



(b) Bajada I/Q

Figura 9: Subida/Bajada I/Q con ventana Blackman-Nuttall.

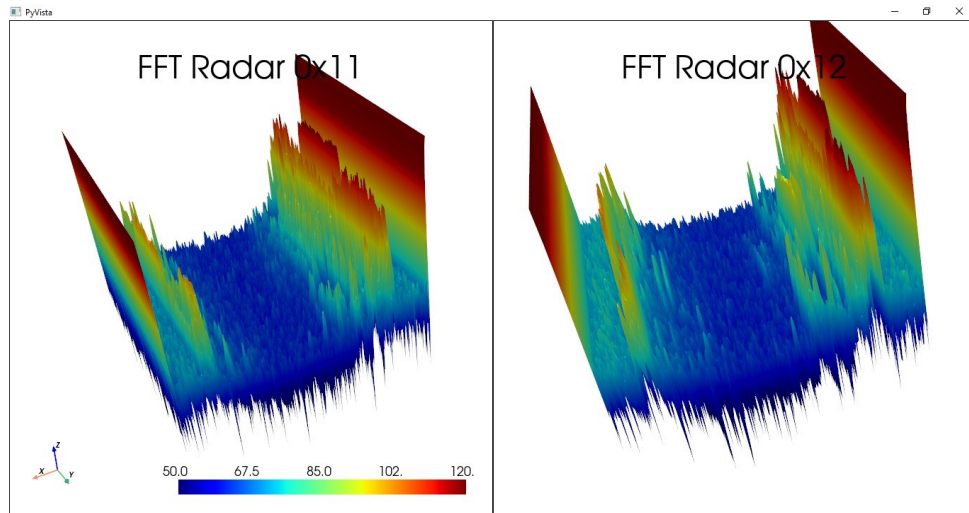
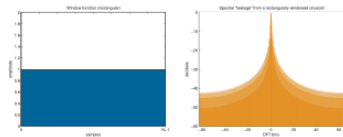
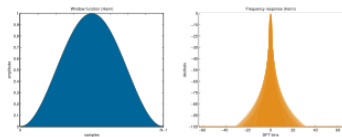


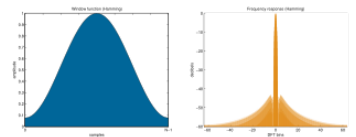
Figura 10: FFT 3D con ventana Blackman-Nuttall.



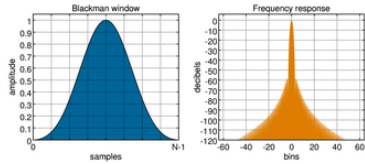
(a) Rectangular



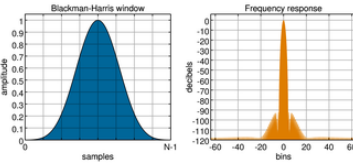
(b) Hanning



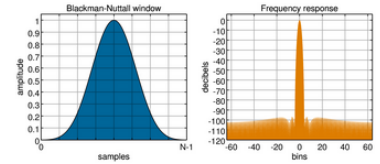
(c) Hamming



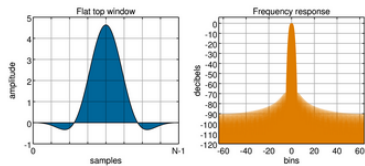
(d) Blackman



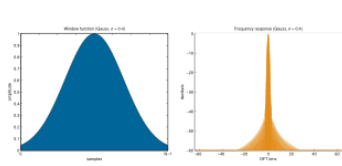
(e) Blackman-Harris



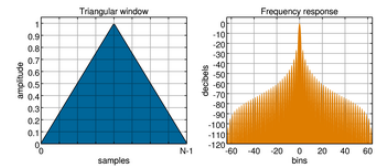
(f) Blackman-Nuttall



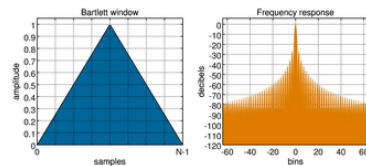
(g) Flat Top



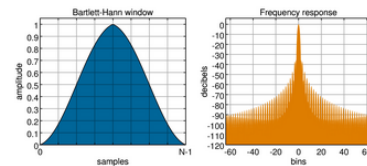
(h) Gauss



(i) Triangular



(j) Bartlett



(k) Bartlett-Hann

Figura 11: Comparativa de los diferentes tipos de ventanas.

- **Zero-Padding:** La resolución visual de una FFT depende del número de puntos de la señal original (ej. 256 o 512). Si la distancia real del objeto cae entre dos puntos calculados, no lo veremos con claridad. El Zero-Padding consiste en añadir ceros al final de la señal temporal antes de la transformada. Matemáticamente, esto no añade nueva información, pero "interpola" la curva espectral, dibujando una gráfica mucho más suave y permitiendo una localización más precisa del pico máximo.

```

1      # Zero Padding:
2      FACTOR_PADDING = 6
3
4      # 3. ZERO PADDING + FFT
5      # a adimos ceros al final de la se al antes de la fft
6      fft_result_up = np.fft.fft(signal_up, n=Nfft_padded)
7      fft_result_down = np.fft.fft(signal_down, n=Nfft_padded)
8
9      fft_g= np.fft.fft(signal_up_g, n=Nfft_padded)
10

```

Listing 4: Código aplicación zero padding

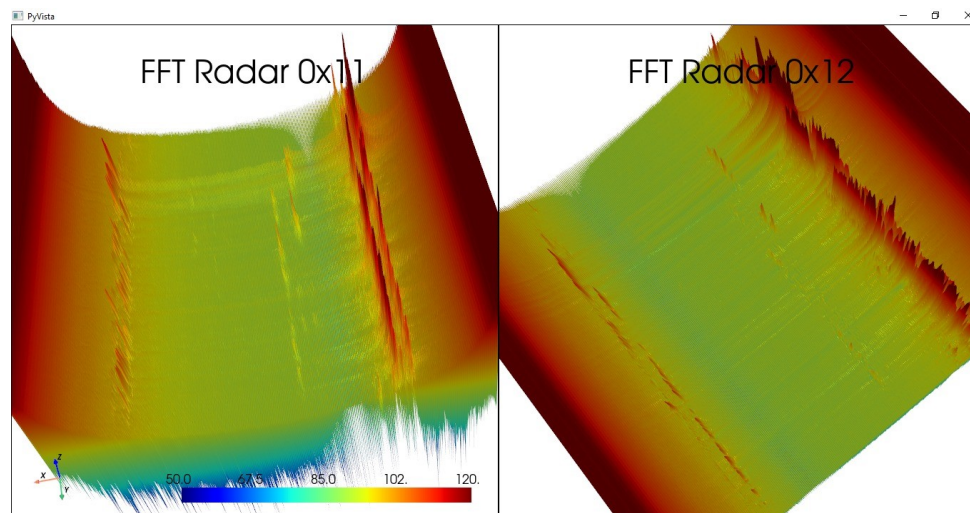


Figura 12: Zero padding 6.

Al implementar estas tres técnicas, resolvimos los problemas que surgieron la semana anterior:

- **Saturación:** Al aplicar el DC Removal, el enorme pico del metro 0 desapareció por completo, dejando a la vista pequeños ecos cercanos que antes estaban ahogados por la energía residual de la antena.
- **Nitidez:** Gracias a la ventana de Nuttall (en mi caso, siempre me decantaba por la ventana Blackman-Nuttall, debido a su gran eficacia), el lóbulo principal de los objetos se estrechó drásticamente. Ahora, si dos objetos viajaran a distancias muy similares, el radar sería capaz de separarlos visualmente.
- **Resolución:** El Zero-Padding convirtió la gráfica inicial dentada y pixelada en una curva espectral continua, facilitando inmensamente el trabajo posterior de los algoritmos de búsqueda de máximos.

4.4. Tercera tarea: *Detección automática y precisión*

Después de conseguir la limpieza de la señal, el siguiente objetivo consistió en automatizar la detección de objetos. Me proporcionaron una nueva carpeta con varias grabaciones de un tramo de carretera en el que podían pasar diversos automóviles. Hasta este punto, la identificación de un vehículo dependía de la inspección visual humana de la gráfica; pero lo que se me pedía ahora era enseñar al sistema a decidir de forma autónoma, y en tiempo real, qué picos correspondían a ecos reales y cuáles eran simple ruido de fondo.

4.4.1. Algoritmo CFAR (*Constant False Alarm Rate*)

La aproximación más básica para detectar un objeto sería establecer un umbral fijo (por ejemplo, determinar que todo lo que supere los 80dB es un coche). Sin embargo, en el mundo real, el ruido electromagnético cambia constantemente debido a varias condiciones (temperatura, lluvia, reflexiones del entorno, etc.). Un umbral fijo provocaría falsas alarmas si el ruido sube, o ignoraría coches si el ruido baja. Para solucionar esto, implementé un algoritmo Cell-Averaging CFAR. En lugar de usar un número estático, el CFAR calcula el umbral de forma dinámica para cada celda basándose en el ruido de su entorno inmediato.

Conceptos Clave del CFAR:

- **CUT** (*Cell Under Test*): La celda que estamos evaluando en ese instante para ver si contiene algún objeto.
- **Guard Cells** (*Celdas de Guarda*): Las celdas inmediatamente adyacentes a la CUT. Se ignoran en el cálculo del ruido para evitar que la propia energía del objeto contamine la media del fondo, evitando que el coche se tape a sí mismo.
- **Training Cells** (*Celdas de Entrenamiento*): Las celdas exteriores que se utilizan para calcular la media aritmética del nivel de ruido local.
- **Factor Alpha** (K): Margen de seguridad que se suma al ruido medio calculado. Si la CUT supera esta suma, se declara una detección oficial.

Al aplicar el CFAR cerca del origen, la energía residual del disparo de la antena elevaba artificialmente la media de las celdas de entrenamiento, provocando que el algoritmo ignorase ecos reales cercanos. Para arreglarlo, modifiqué el código justo antes de ejecutar el CFAR: forcé el valor de los primeros 10 bins (metros iniciales) a -180dB. Esto garantizó que el cálculo del ruido de fondo a partir del metro 11 fuera puro y permitiera la detección de objetos cercanos.

```
1  def aplicar_cfar(signal_dB, num_gap, num_cut, margen_dB):
2      """
3      Aplicamos CFAR a una se al 1D en dB
4      - signal_dB: Array con la magnitud de la FFT
5      - num_gap: N mero de celdas de guarda a cada lado
6      - num_cut: N mero de celdas bajo prueba, las que hay que analizar si son
7      objetivos o no
8      - margen_dB: Margen de seguridad, umbral sobre el ruido
9
10     Retorna:
11     - threshold: umbral din mico calculado
12     - detecciones: array booleano, true cuando haya objetivo
13     """
14     n = len(signal_dB)
15     threshold = np.full(n, 200.0) # Iniciamos con un valor alto para no detectar
16     bordes
17     detecciones = np.zeros(n, dtype=bool)
```

```

17 # Tama o total de la ventana a un lado
18 window_side = num_gap + num_cut
19
20 # Recorremos la se al (saltando los bordes donde no cabe la ventana)
21 for i in range(window_side, n - window_side):
22
23     # 1. Sacar el vecindario (Izquierda y Derecha), saltando las celdas gap
24     gap_left = signal_dB[i - window_side : i - num_gap]
25     gap_right = signal_dB[i + num_gap + 1 : i + window_side + 1]
26
27     # 2. Calcular el Nivel de Ruido (Promedio del vecindario)
28     noise_level = np.mean(np.concatenate((gap_left, gap_right)))
29
30     # 3. Calcular el Umbral (Ruido + margen)
31     thresh_val = noise_level + margen_dB
32     threshold[i] = thresh_val
33
34     # 4. Es el pico mayor que el umbral
35     if signal_dB[i] > thresh_val:
36         detecciones[i] = True
37
38 return threshold, detecciones

```

Listing 5: Función algoritmo CFAR

```

1 # --- DETECCION DE OBJETOS (TU CFAR SOBRE LA FOTO MEDIA) ---
2 espectro_medio_lineal = np.mean(np.abs(fft_result), axis=0)
3 espectro_medio_lineal[:15] = 0 # Ignorar el grito de la propia antena
4 espectro_medio_db = 20 * np.log10(np.maximum(espectro_medio_lineal, 1e
-9))
5
6 # 1. Tu Guardia de Seguridad revisa la habitaci n (Ejemplo: 2 de gap,
7 12 de vecinos, 8dB de exigencia)
8 umbral_cfar, zonas_detectadas = aplicar_cfar(espectro_medio_db, num_gap
=2, num_cut=12, margen_dB=8.0)
9
10 # 2. Buscamos todas las cumbres matem ticas (las puntas de las
11 monta as)
12 picos_crudos, _ = find_peaks(espectro_medio_db, distance=10)
13
14 # 3. Trabajo en equipo: Nos quedamos SOLO con las cumbres que caigan
15 dentro de una zona que tu CFAR haya marcado como 'True' (Objetivo Real)
16 indices_objetos = [p for p in picos_crudos if zonas_detectadas[p] ==
True]
17
18 print(f" Detectados {len(indices_objetos)} objetos usando el CFAR
propio.")

```

Listing 6: Uso CFAR

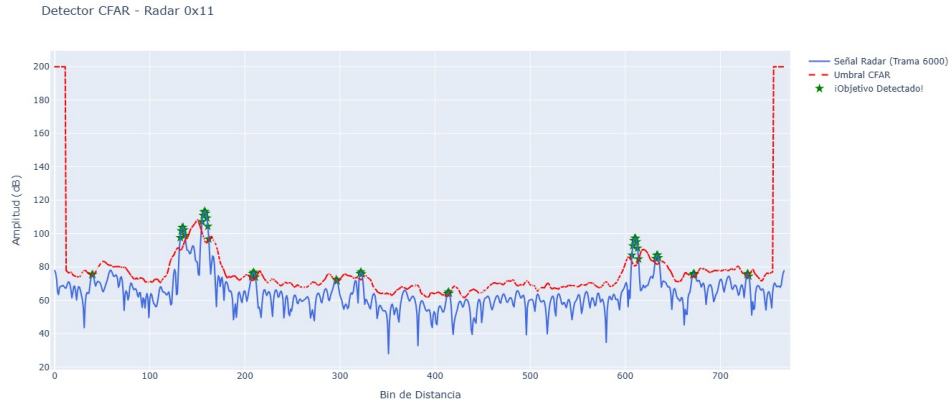


Figura 13: CFAR.

4.4.2. Precisión Sub-bin e Interpolación Gaussiana

El radar divide el espacio en baldosas o bins (por ejemplo, cada baldosa mide 0.075m). Cuando el CFAR detecta el pico máximo, nos devuelve un índice entero (ej. bin 62). Si multiplicamos 62×0.075 , obtenemos 4.65m. El problema es que el objeto real podría estar en 4.68m o en 4.62m. Estar limitados por el tamaño de la baldosa generaba un seguimiento escalonado, lo cual es inaceptable para calcular velocidades precisas a posteriori. Para superar esta limitación física del hardware y obtener distancias continuas (milimétricas), implementé un algoritmo de interpolación Gaussiana sobre los tres puntos más altos del pico detectado.

El algoritmo toma la celda máxima (Y_{max}), su vecina izquierda (Y_{-1}) y su vecina derecha (Y_{+1}) y calcula el vértice exacto de la parábola que pasa por esos tres puntos mediante la siguiente formulación matemática en código:

```

1  def calcular_distancia_precisa(espectro_dB, resolucion_por_bin):
2  """Calcula la distancia exacta (sub-bin) usando interpolación gaussiana."""
3  idx_pico = np.argmax(espectro_dB)
4  val_pico = espectro_dB[idx_pico]
5
6  if idx_pico <= 0 or idx_pico >= len(espectro_dB) - 1:
7      return idx_pico * resolucion_por_bin
8
9  val_izq = espectro_dB[idx_pico - 1]
10 val_der = espectro_dB[idx_pico + 1]
11
12 numerador = np.log(val_izq) - np.log(val_der)
13 denominador = np.log(val_izq) - 2 * np.log(val_pico) + np.log(val_der)
14
15 if denominador == 0: return idx_pico * resolucion_por_bin
16
17 delta = 0.5 * numerador / denominador
18 return (idx_pico + delta) * resolucion_por_bin

```

Listing 7: Función interpolación Gaussiana

Al extraer las medidas de trayectoria de un vehículo, la diferencia fue drástica. Las medidas brutas (sin Gauss) generaban una trayectoria en forma de escalera, manteniendo el mismo valor durante varios milisegundos hasta saltar a la siguiente baldosa. Al aplicar la técnica con Gauss, la trayectoria se convirtió en una línea continua y fluida, reflejando el movimiento real y orgánico del vehículo en el espacio continuo.

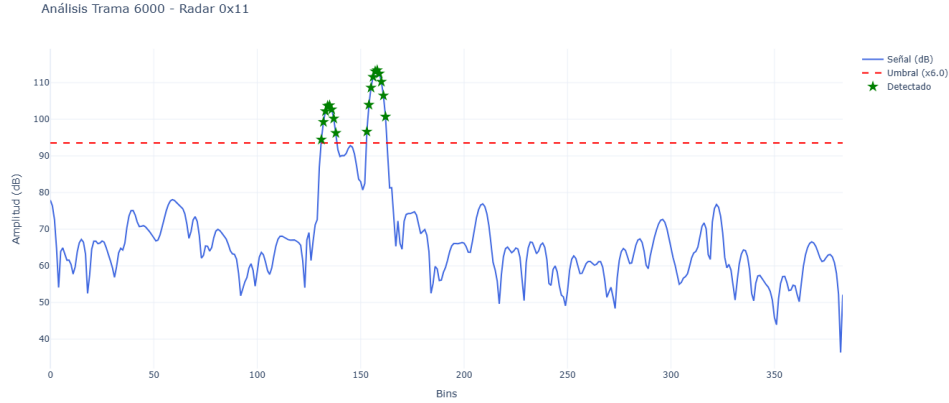


Figura 14: Umbral adaptativo con interpolación gaussiana.

4.5. Cuarta tarea: *Análisis de Micro-Doppler y Fases de Vibración*

Una vez que tenía asegurada la detección de la posición de los objetos mediante CFAR y la interpolación Gaussiana, el siguiente paso era la extracción de biometría o firmas mecánicas, analizar los micro-movimientos (*Micro-Doppler*) de los objetivos detectados. Para esta tarea, me proporcionaron dos conjuntos de datos diametralmente opuestos para poner a prueba la flexibilidad del sistema:

- Escenario Portuario (Estático): Datos de un barco mercante atracado, cuyo único movimiento era el balanceo del mar y la vibración interna de sus motores.
- Escenario Freeflow (Dinámico): Datos de tráfico real con vehículos desplazándose a altas velocidades continuas.

4.5.1. La Fase de la Señal Compleja

Para extraer la vibración, había que abandonar la magnitud de la FFT (que nos daba la distancia en metros) y centrar la atención en la fase del número complejo calculado.

- La Fase (ϕ): Cuando el radar capta el rebote, recibe un número complejo ($A + bj$). Mediante la función trigonométrica arco-tangente ($\text{atan2}(b, a)$), obtenemos el ángulo o fase de la onda en ese instante exacto.
- Significado: Si un objeto está estático, su fase entre dos disparos del radar debería ser idéntica. Si el objeto vibra un milímetro, ese milímetro provoca un cambio minúsculo en la fase de la onda de radio recibida. Leyendo cómo cambia esa fase milisegundo a milisegundo, podemos escuchar la vibración del motor del barco o del coche.

4.5.2. Implementación: Desenrollado (Unwrap) y Estabilización

Para poder extraer la fase se debía tratar de un forma específica, debido al problema de Wrapping (*Salto de Fase*): El ángulo de un número complejo está acotado entre $-\pi$ y $+\pi$ radianes (o -180° y $+180^\circ$). Si un objeto se mueve y su fase supera los 180° , la matemática la reinicia a -180° . En la gráfica, esto se visualizaba como un electrocardiograma roto, lleno de líneas verticales abruptas que destruían cualquier intento de leer una vibración continua.

Entonces, para solucionarlo había que hacer Unwrap. Se implementó la función matemática `np.unwrap`. Este algoritmo detecta los saltos artificiales mayores a π y los compensa sumando o restando 2π , reconstruyendo la onda de movimiento en una curva continua e ininterrumpida. Además, también había que eliminar la tendencia: una vez continua, la onda solía tener una

gran inclinación (debido a la velocidad del coche o al viento empujando el barco). Para aislar solo la vibración microscópica, apliqué la función *scipy.signal.detrend*, que calcula la línea recta de esa inclinación y se la resta a la señal, dejando la onda de vibración perfectamente horizontal y centrada en el cero.

```

1 # 1. Extracción de la fase bruta del bin detectado (ej. Bin 62)
2 fase_cruda = np.angle(fft_result[:, bin_max])
3
4 # 2. Desenrollado para evitar saltos de -PI a +PI
5 fase_desenrollada = np.unwrap(fase_cruda)
6
7 # 3. Estabilización: Eliminamos la deriva de velocidad para dejar solo la
   vibración
8 fase_limpia = signal.detrend(fase_desenrollada)

```

Al analizar la fase limpia de diferentes vehículos en la carretera, me percaté de una anomalía: las ondas de vibración de dos coches distintos, que pasaban en momentos diferentes, mostraban exactamente los mismos picos y valles en los mismos segundos. Era imposible que dos motores distintos vibraran de forma sincronizada. La fase extraída no era la del coche, sino la vibración del trípode del propio radar movido por el viento y la dilatación térmica de los cables. Al moverse el radar, parecía que todo el universo frente a él vibraba al mismo ritmo. La solución fue implementar la Técnica de Referencia de 2 Celdas. Seleccioné una baldosa muy cercana al radar, que se sabía que siempre correspondía a suelo estático (ej. el metro 2). Saqué la fase del coche y la fase del suelo de forma independiente.

Al restar la Fase del Coche menos la Fase del Suelo Estático, todo el ruido mecánico común (vibración del trípode, ruido de fase interno del chip) se canceló matemáticamente. El resultado final fue la extracción, por primera vez, de la firma de vibración pura e individual del objetivo.

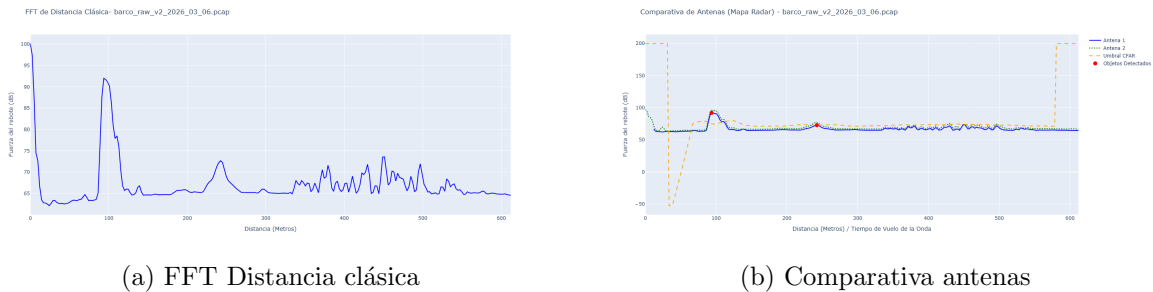


Figura 15: Resultados medidas Barco

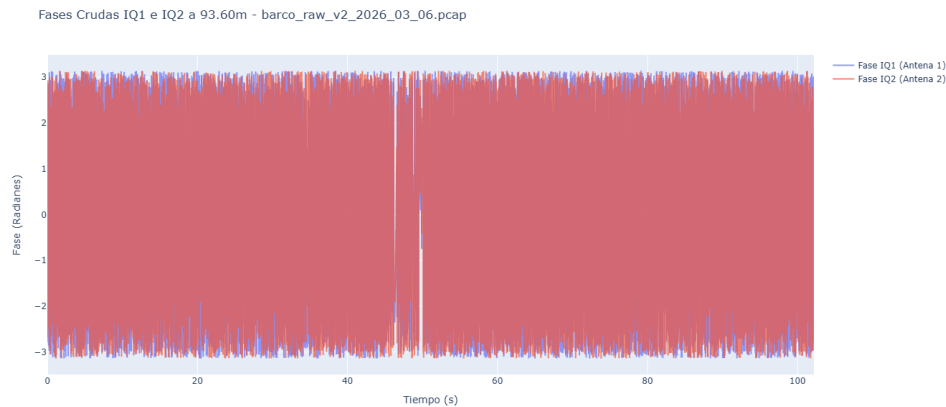
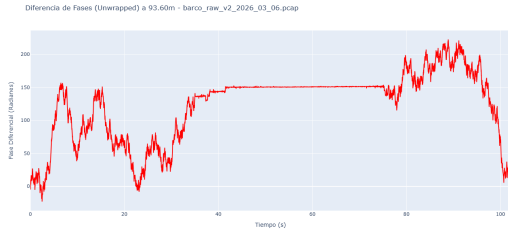
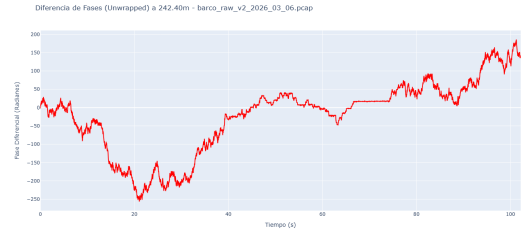


Figura 16: Fases crudas IQ1/IQ2 Barco

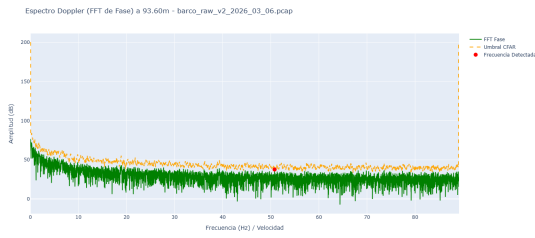


(a) Diferencia de fases 1

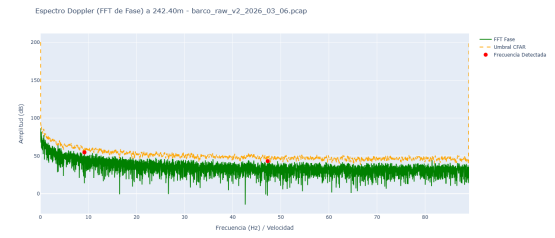


(b) Diferencia de fases 2

Figura 17: Comparativa de fases. Unwrap



(a) Efecto Doppler Objeto 1



(b) Efecto Doppler Objeto 2

Figura 18: Comparativa Efecto Doppler (FFT Distancia) detección de Barcos

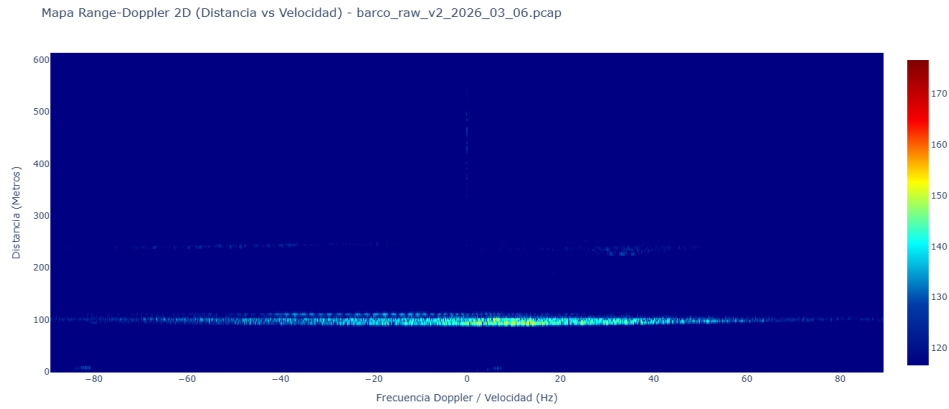


Figura 19: Mapa de calor detección Barco

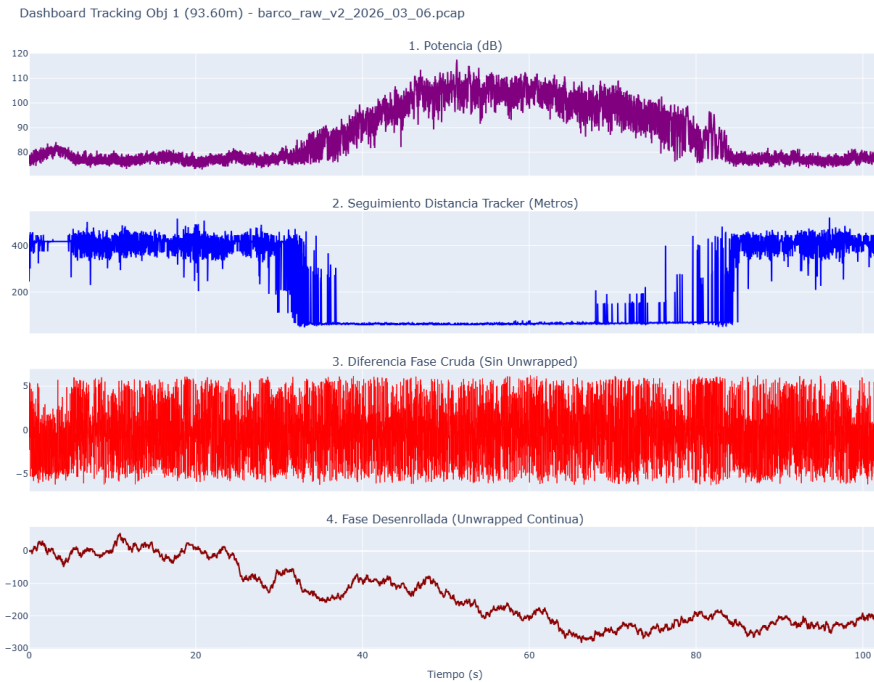


Figura 20: Comparativa Barco

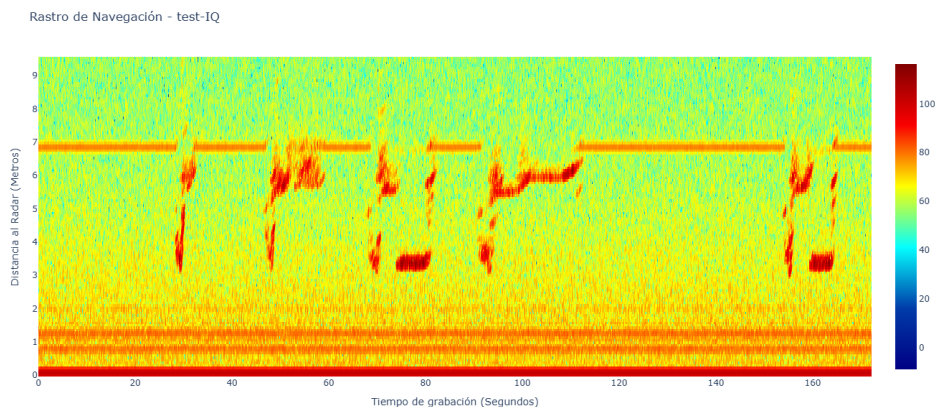


Figura 21: Mapa de calor

4.5.3. Última tarea: *Filtro de Kalman*

Para la última semana de mis prácticas, me asignaron una tarea enfocada en la investigación y formación avanzada: estudiar y comprender matemáticamente el **Filtro de Kalman**. Para ello, utilicé como material de referencia principal el manual interactivo *kalmanfilter.net* (Alex Becker).

Aunque el objetivo inicial era puramente teórico (entender el concepto para futuras aplicaciones), tomé la iniciativa personal de intentar programar el algoritmo desde cero en Python e integrarlo en el código de radar que habíamos estado desarrollando las semanas anteriores.

4.5.4. ¿Qué es el Filtro de Kalman y por qué es necesario?

En el mundo del procesamiento de señales, los sensores nunca son perfectos; el radar siempre tiene un margen de error térmico o ambiental (ruido). El Filtro de Kalman es un algoritmo

matemático recursivo que estima el estado verdadero (y oculto) de un sistema dinámico a partir de una serie de mediciones incompletas y ruidosas.

En lugar de confiar ciegamente en lo que el radar ve en cada milisegundo, el filtro le otorga inercia al sistema mediante un ciclo continuo de dos pasos:

1. **Fase de Predicción:** Utilizando las leyes de la cinemática (velocidad $\times$ tiempo), el filtro calcula matemáticamente dónde debería estar el coche en el próximo instante, basándose en su trayectoria pasada.
2. **Fase de Actualización:** El radar realiza una nueva medición. El algoritmo calcula la Innovación (la diferencia entre lo que predijo y lo que midió el radar). Mediante una variable dinámica llamada **Ganancia de Kalman (K)**, el sistema decide matemáticamente de quién se fía más: si la medición es muy ruidosa, confía en su predicción; si la medición es muy clara, corrige su trayectoria para ajustarse a la realidad.

El intento de trasladar la teoría matemática de *kalmanfilter.net* a nuestro flujo de datos I/Q reales tuvo dos grandes problemas:

- La teoría explicaba el filtro mediante variables abstractas, pero para que funcionara en nuestro código, era necesario construir un modelo matricial unidimensional de segundo orden (que calculara Posición y Velocidad simultáneamente). Por lo que, para llevarlo a cabo, diseñé y programé las matrices del sistema en NumPy. Definí la Matriz de Medición (H) para indicarle al algoritmo que el radar solo nos daba metros (posición), obligando a Kalman a deducir la velocidad por sí mismo. Además, sistematicé los parámetros de incertidumbre: establecí una matriz de Ruido de Medida (R) equivalente a 1,5 metros de error típico del radar, y ajusté el Ruido de Proceso (Q) dependiendo de si seguíamos a un barco (movimiento predecible, Q baja) o a un coche en carretera (aceleraciones bruscas, Q alta).
- Al probar mi código recién programado con los archivos de tráfico (*Freeflow*), el filtro funcionaba perfectamente hasta que los coches se acercaban a menos de 4 metros del radar. En ese instante, el algoritmo colapsaba, arrojando una velocidad deducida imposible de -600 km/h. Al analizar el fallo, descubrí que la Ventana de Búsqueda del algoritmo estaba rozando el metro cero. El filtro veía el inmenso destello electromagnético de la antena y, al ser mucho más potente que el coche, lo "secuestraba", haciendo que el tracker saltara de 4 metros a 0 metros en un milisegundo. Para evitar este secuestro, implementé un límite físico en la programación de la ventana de validación. Esta simple restricción geométrica obligaba al algoritmo a ignorar cualquier señal proveniente de la antena, forzando al filtro a mantener la atención sobre el vehículo y logrando, finalmente, una gráfica de velocidad completamente estable e ininterrumpida.

```
1 tiempo_segundos = np.arange(len(fft_result1)) * t_chirp
2
3     for num, obj_bin in enumerate(indices_objetos):
4         distancia_inicial = obj_bin * bin_padded_res
5         print(f"      Iniciando Kalman - Objeto {num+1} a {distancia_inicial
6           :.2f}m...")
7
8         posiciones_crudas_radar = []
9         posiciones_limpias_kalman = []
10        velocidades_kalman = []
11        potencias_tiempo = []
12        fases_tiempo_rx1 = []
13        fases_tiempo_rx2 = []
14
15        # ---INICIALIZACION MATRICES KALMAN ---
```

```

15     X = np.array([[distancia_inicial], [0.0]]) # Estado: [Posición,
    Velocidad]
16     F = np.array([[1.0, t_chirp], [0.0, 1.0]]) # Transición (Física)
17     H = np.array([[1.0, 0.0]]) # Medición (Solo leemos
    posición)
18     P = np.array([[1.0, 0.0], [0.0, 10.0]]) # Dudas iniciales
19     Q = np.array([[0.001, 0.0], [0.0, 0.1]]) # Ruido del coche/barco
20     R = np.array([[2.0]]) # Ruido del radar
21     I = np.eye(2) # Matriz identidad
22
23     # ---BUCLE DE SEGUIMIENTO TEMPORAL ---
24     for t in range(len(fft_result1)):
25         # Paso A: PREDICCIÓN
26         X_pred = np.dot(F, X)
27         P_pred = np.dot(F, np.dot(P, F.T)) + Q
28
29         bin_pred = int(X_pred[0, 0] / bin_padded_res)
30
31         # Paso B: MEDIDA DEL RADAR
32         inicio = max(55, bin_pred - 10)
33         fin = min(len(espectro_medio_db1), bin_pred + 5)
34
35         ventana_fft = fft_result1[t, inicio:fin]
36         mag_t_db = 20 * np.log10(np.maximum(np.abs(ventana_fft), 1e-9))
37
38         idx_local = np.argmax(np.abs(ventana_fft))
39         idx_real = inicio + idx_local
40
41         Z = np.array([[idx_real * bin_padded_res]]) # Lo que ve el radar
42
43         # Paso C: ACTUALIZACIÓN KALMAN (La Ganancia)
44         Y = Z - np.dot(H, X_pred)
45         S = np.dot(H, np.dot(P_pred, H.T)) + R
46         K = np.dot(P_pred, np.dot(H.T, np.linalg.inv(S)))
47
48         X = X_pred + np.dot(K, Y)
49         P = np.dot((I - np.dot(K, H)), P_pred)
50
51         # Guardamos datos para pintar las gráficas
52         posiciones_crudas_radar.append(Z[0, 0])
53         posiciones_limpias_kalman.append(X[0, 0])
54         velocidades_kalman.append(X[1, 0] * 3.6) # En km/h
55         potencias_tiempo.append(mag_t_db[idx_local])
56
57         # 2 CELDAS: Comparamos el barco con el suelo (baldosa 8)
58         idx_ref_fija = 8
59         fases_tiempo_rx1.append(np.angle(fft_result1[t, idx_real]) - np.
    angle(fft_result1[t, idx_ref_fija]))
60         fases_tiempo_rx2.append(np.angle(fft_result2[t, idx_real]) - np.
    angle(fft_result2[t, idx_ref_fija]))
61
62         # Fase diferencial y Unwrap
63         diferencia_fase_cruda = np.array(fases_tiempo_rx1) - np.array(
    fases_tiempo_rx2)
64         fase_limpia = np.unwrap(diferencia_fase_cruda)

```

Listing 8: Algoritmo de Kalman

En el escenario portuario, para seguir a un barco (un objeto de gran masa y movimientos lentos), se configuró una Q muy baja. Esto indicó al filtro que el objetivo era altamente predecible, logrando ignorar destellos de ruido y manteniendo una trayectoria rígida y estable. En el escenario de Tráfico (Freeflow), para los vehículos en la carretera, capaces de acelerar o frenar bruscamente, se incrementó el valor de Q . Esto hizo que la Ganancia de Kalman diera más peso

a las mediciones recientes del radar, permitiendo al tracker seguir las maniobras ágiles de los coches sin quedarse rezagado.

5. Relación con el Grado

El desarrollo de este proyecto de procesamiento de señales de radar ha supuesto la aplicación práctica de múltiples conceptos teóricos y herramientas matemáticas adquiridas a lo largo de mis estudios en el **Grado en Matemáticas**. Lejos de ser un trabajo puramente informático, la naturaleza del radar FMCW requiere un profundo entendimiento físico y matemático.

A continuación, detallaré la vinculación directa de las tareas realizadas con las competencias específicas de diversas asignaturas del plan de estudios:

5.1. Álgebra Lineal y Geometría

El pilar fundamental del álgebra matricial se ha aplicado de forma transversal durante todo el proyecto, culminando en la programación del **Filtro de Kalman**.

- **Cálculo Matricial y Espacios Vectoriales:** El estado del sistema (la posición y velocidad del vehículo) se definió como un vector de estado X . Las operaciones de predicción y corrección del filtro requirieron el dominio de multiplicaciones de matrices multidimensionales (Matriz de Transición F , Matriz de Observación H), así como el cálculo de matrices transpuestas e inversas (esenciales para obtener la Ganancia de Kalman K).
- **Vectores y Geometría en 3D:** Durante la primera fase del proyecto, el uso del `3d_visualizer` requirió la comprensión de sistemas de coordenadas cartesianas y transformaciones espaciales para representar correctamente las nubes de puntos detectadas por el radar en un entorno tridimensional.

5.2. Análisis Matemático y Variable Compleja

El tratamiento de la señal electromagnética es, en esencia, un problema de análisis matemático complejo aplicado a la física ondulatoria.

- **Números Complejos (Señal I/Q):** La telemetría bruta del radar no se procesa mediante números reales, sino mediante datos en Fase (I) y Cuadratura (Q), conformando números complejos de la forma $z = I + Qj$. Comprender el plano complejo fue vital para extraer la Magnitud (módulo $|z|$) para el cálculo de distancias, y el Ángulo (argumento $\text{atan2}(Q, I)$) para el cálculo de vibraciones y Micro-Doppler.
- **Análisis de Fourier:** La piedra angular técnica de las prácticas ha sido la **Transformada Rápida de Fourier (FFT)**. Los conocimientos teóricos sobre series y transformadas de Fourier permitieron entender el paso del dominio del tiempo continuo (la señal de batido) al dominio de la frecuencia discreta. Además, se aplicaron conceptos de interpolación polinómica (Interpolación Gaussiana) para calcular raíces y vértices parabólicos, logrando así resolución sub-bin en las distancias.

5.3. Estadística, Probabilidad y Optimización

La detección de objetos en un entorno ruidoso (como la calle o un puerto) es un problema clásico de toma de decisiones bajo incertidumbre.

- **Distribuciones y Probabilidad:** La implementación del algoritmo **CFAR (Constant False Alarm Rate)** se fundamenta en la teoría de la probabilidad. Para determinar si una señal es un coche o es ruido, el código calcula dinámicamente la media aritmética del

ruido de fondo (asumiendo una distribución exponencial o de Rayleigh) y establece un umbral probabilístico para garantizar una tasa constante de falsas alarmas.

- **Matrices de Covarianza:** En el Filtro de Kalman, la incertidumbre del sistema se modela matemáticamente a través de las matrices de covarianza de error (P), de ruido del proceso (Q) y del sensor (R). Ajustar estos parámetros estadísticos fue lo que permitió diferenciar el comportamiento inercial de un barco frente a las aceleraciones erráticas de un coche.

5.4. Programación Estructurada y Algoritmia

El traslado de la matemática teórica a un producto funcional ha requerido el uso de competencias algorítmicas y lógicas.

- **Eficiencia Computacional:** Como he comentado ya, el manejo de millones de puntos de datos generaba desbordamientos de memoria RAM. La aplicación de buenas prácticas de programación, el uso eficiente de bucles lógicos condicionales y la optimización de los tipos de datos (reducción forzada de `float64` a `float32`) permitieron que los complejos algoritmos matemáticos se ejecutaran en tiempo real, validando los conocimientos de algoritmia y estructuras de datos adquiridos en la carrera.

6. Conclusiones

Durante mi estancia en la empresa, he podido comprobar de primera mano cómo la matemática pura y aplicada es el motor de la industria tecnológica, concretamente en el sector de los sensores y la telemetría. He comprendido que los conceptos teóricos estudiados en la facultad no son entes abstractos, sino herramientas empresariales de alto valor que permiten a una compañía medir el tráfico en tiempo real, optimizar infraestructuras portuarias o evaluar la salud mecánica de un motor. He adquirido una visión clara de cómo el perfil del matemático es esencial en el sector privado para resolver problemas analíticos que escapan a la ingeniería tradicional.

La transición de los datos limpios e ideales de un entorno académico al ruido electromagnético y la incertidumbre de un entorno físico real ha supuesto el mayor hito de aprendizaje de estas prácticas. He aplicado cálculo de probabilidades (mediante el algoritmo CFAR) para lidiar con el ruido de fondo impredecible de la calle. Asimismo, he utilizado herramientas de cálculo numérico (Interpolación Gaussiana) para superar las limitaciones físicas del hardware del radar, y he implementado sistemas probabilísticos y matriciales (Filtro de Kalman) para estabilizar trayectorias caóticas de vehículos.

Todo este despliegue algorítmico se ha materializado gracias al dominio de **Python**, una competencia clave cimentada en la asignatura de *Tecnología de la Programación* y en la vertiente práctica del Grado, lo que ha permitido ejecutar el proyecto con total solvencia técnica. El resultado ha sido la transformación de variables matemáticas abstractas en métricas reales y tangibles: metros de distancia, kilómetros por hora y Hercios de vibración.

Cabe resaltar el ambiente de trabajo que he experimentado y el gran equipo que hay detrás de esta empresa. Desde el primer momento mi tutor estuvo totalmente pendiente de que comprendiese bien todos los conceptos, proporcionándome multitud de material didáctico, demostraciones y aplicaciones reales, explicaciones suyas, resolviendo mis dudas, orientándome en las dificultades... Me han hecho partícipe en otros proyectos suyos más prácticos para mejorar la comprensión y adentrarme más aún en el mundo del radar. Todo esto, ha hecho que mi estancia durante las prácticas resultase muy agradable y motivadora, culminando en una experiencia muy provechosa.

Puedo concluir expresando que ha sido una gran y enriquecedora experiencia, tanto a nivel personal como académicamente. Mi objetivo principal al realizar prácticas externas era tener la oportunidad de poder aplicar todos los conocimientos adquiridos en el Grado a algún proyecto

real, alejarme de los resultados teóricos y ver los impactos reales. Gracias a esta oportunidad, me he dado cuenta de todos los conocimientos que he adquirido durante estos años y cómo se pueden aplicar en el mundo real sin limitarse a resultados académicos teóricos; además del gran sentido crítico y analítico de problemas que te proporciona el Grado, así como la perseverancia y el juicio crítico hasta conseguir el objetivo final, formándote de todas las formas posibles hasta conseguirlo.